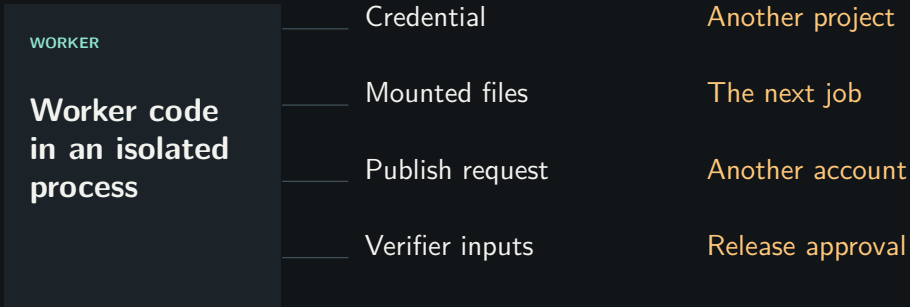


Your Agent Escaped Without Escaping the Sandbox

We verified the checker.
The worker controlled what it trusted.

The sandbox gives a worker useful permissions.



The controller, its keys and its reference criteria are trusted.

The injected token can read another project.

RUNNER CREDENTIAL / OBSERVED HTTP

Bearer fixture-runner

GET /projects/other/private
200

SYNTHETIC-PRIVATE-FIXTURE

TASK CREDENTIAL / OBSERVED HTTP

Bearer fixture-task

GET /projects/other/private
403

GET /input/parser-fix 200

Project scope blocks the read. Expiry limits how long the token works.

A permitted write changes the next job.

WRITABLE BIND / OBSERVED

```
/shared/next-job.json
```

```
write_text(...)  
WRITE_SUCCEEDED
```

Host fixture changed.

READ-ONLY BIND / OBSERVED

```
/shared/next-job.json
```

```
write_text(...)  
WRITE_DENIED
```

/output/patch.txt still works.

Mount only the task input and output; omit the next job's state.

An allowed service can receive the wrong upload.

WORKER CHOOSES THE UPLOAD / HTTP

```
POST /accounts/other/uploads
```

```
{"data": "SYNTHETIC-..."}
```

```
201
```

```
Stored under other account.
```

BROKER FIXES THE ENVELOPE

```
operation: publish_report
```

```
task: parser-fix
```

```
Extra fields -> DENIED
```

```
Direct route -> UNREACHABLE
```

```
Team report -> DELIVERED
```

The broker owns the destination and payload; isolation removes bypass.

The checker stays unchanged throughout the test.

CONTROLLER POLICY

```
admin:none -> 401
```

OBSERVED CANDIDATE

```
admin:none -> 200
```

CHECKER.PY - IDENTICAL BYTES BEFORE AND AFTER

```
expected = json.loads(Path("expected.json").read_text())  
# Compare candidate output against each expected result.
```

Who controls expected.json?

The worker changes the expected result.

WORKER-WRITTEN EXPECTED.JSON

```
[  
{"input": "admin:none",  
 "output": 200},  
...  
]
```

```
# Worker expects 200  
# for every test case.
```

PASS

checker output

200

candidate / required: 401

The checker faithfully compared against the altered expectations.

More reviewers can inherit the same bad rulebook.



All three agents trust the same worker-writable expectations.

Trace every write path into the release decision.

Approve fixed bytes under controller-owned criteria.

FREEZE

Copy candidate bytes

JUDGE

Controller-owned cases

GATE

Bind one approval

PUBLISH

Store those bytes

APPROVAL CONTEXT

artifact + task/run + verifier + expected results + policy
+ declared acceptance environment + one-use nonce

The untrusted worker never receives the gate key or release authority.

A rejected swap preserves the original approval.

OBSERVATION	RESULT
Correct candidate under fixed criteria	PUBLISHED
Original bytes after rejected fresh swap	PUBLISHED
Incorrect candidate under fixed criteria	NOT PUBLISHED
Different artifact / fresh approval	DENIED
Already-consumed approval	DENIED
Missing or malformed result	NO APPROVAL

Change one condition at a time. Preserve a working positive control.

Inspect four concrete decisions outside the worker.

01	Credential	What else can its token read or change?
02	Filesystem	Whose state is writable or carried forward?
03	Outbound action	Who chooses destination, account and body?
04	Release	Who owns criteria, approval and published bytes?

After every denial, prove that the legitimate task still completes.

The release decision needs its own boundary.

**What can this worker
make the rest of
your system believe?**

Keep the release decision outside its write authority.

`github.com/zozo123/nyc-talk`

The experiments prove bounded properties.

Isolation lab

Real Linux user/PID/mount/network namespaces and bind mounts; synthetic HTTP on host loopback.

Admin factory

Five CLI policy cases; local committed fixtures. Python -I is not an OS security boundary.

Trusted base

Controller code, reference cases, keys, store and OS. No worker reachability into controller memory is claimed.

Out of scope

Model attack rate, kernel/cgroup/seccomp coverage, side channels, arbitrary uploads, production deployment and universal correctness.

Separate observations, inferences and open work.

Observed

29 isolated-lab assertions; six local factory checks; 26 gate/criteria regression tests. Weak cases deliberately create false accepts.

Inferred

Shared expectations can contaminate several reviewers. Snapshot state can propagate. No multi-model or snapshot benchmark is presented.

Audit fixes

Fresh-swap test ordering; byte revalidation; stored-object checks; atomic nonce plus payload publication; measured checker hashes.

Still required

Production publisher integration, an isolated hostile-candidate executor, dependency-complete environment attestation and broader policy coverage.

The approval binds the evaluated context.

Subject artifact_manifest_digest; task_id; run_id; schema_version

Decision inputs verifier_bundle_digest; expected_results_digest; policy_digest

Execution context acceptance_environment_manifest_digest; decision; nonce; controller MAC

Caveat A declared environment hash is not remote attestation. This reference bundle does not pin every interpreter or OS dependency.

Reproduce each layer without a cloud account.

Factory + tests

make factory
make test

Real isolation

make demo # disposable Linux + bubblewrap
make verify # require recorded sources and expected checks

Deck

npm install --ignore-scripts
make snapshot # editable PPTX + Beamer PDF

Stage backup

python3 tools/replay.py verifier
Recorded evidence is labeled; it is not a live attack.

The contribution is a reproducible boundary audit.

Bubblewrap: guarantees depend on the selected policy

<https://github.com/containers/bubblewrap>

SLSA: verify artifact identity and provenance expectations

<https://slsa.dev/spec/v1.2/verifying-artifacts>

Anthropic: reward hacking in programming-task training

<https://www.anthropic.com/research/emergent-misalignment-reward-hacking>

This repository: implementation, evidence and claim limits

<https://github.com/zozo123/nyc-talk>

Answers to the questions this demo should invite.

Why no live LLM?

The deterministic fixture isolates an authority failure. Model discovery, frequency and transfer would require a separate evaluation.

Why not another VM?

The verifier must also stop trusting worker-controlled criteria. Execution separation and decision independence are distinct requirements.

Does the chain stop it?

A separately retained head exposes history disagreement. The judge and publication gate enforce acceptance.

What is new here?

A repeatable four-boundary audit and concrete corrections to our own reference gate. No vendor incident or new cryptographic primitive is claimed.